

# PRD — Department Coordination Dashboard (Next Generation)

---

Reference document for building the React + Firebase department dashboard. Version 1.0 · Supersedes the single-team "Coordinator Management (Malmö 16)" app.

---

## 0. How to Use This Document

---

This is the build specification for the new app. It is written so it can be:

1. **Pasted into an AI coding assistant** as the source of truth for generating the React project, and
2. **Kept open in VS Code** as a reference while building.

It assumes the old single-team app as background context but **defines a new app** — do not carry over old structure unless this document explicitly says to. Where this document and the old app disagree, **this document wins**.

The companion file `firestore.rules` is the complete, ready-to-deploy security ruleset and is referenced throughout Section 9.

---

## 1. Purpose & Vision

---

A real-time internal coordination tool for a growing Swedish public-sector department. It replaces manual tracking (whiteboards, spreadsheets, chat) with a shared live dashboard where every team member sees the same state simultaneously, with no page refresh.

The previous version tracked **one** team. This version tracks a **whole department of many teams**, gives the department a single weekly goal that all teams contribute toward, and introduces **real user accounts** with two roles instead of a shared admin PIN.

The central job of the app: **give coordinators a clear overview of the department's work, while letting every member contribute their numbers and manage their own schedule with minimal friction**.

---

## 2. Goals & Non-Goals

---

### Goals

- One shared, real-time view of the whole department's weekly progress.
- A single department-level weekly goal for **Beredningar**, fed by all teams.
- Per-team case tracking (Beredningar and Beslut counters).
- A weekly schedule built on **groups** that rotate tasks automatically.
- Simple absence reporting (two types) that members manage themselves.
- Two coordinators who can administer the whole department.
- Real authentication and enforced, role-based permissions.
- The department can grow autonomously — coordinators add teams and members without a developer.

## Non-Goals (explicitly out of scope for v1)

- A separate read-only "manager" role (coordinators report upward manually).
- Per-team coordinators (one or two coordinators cover the whole department).
- The decision-deadline **calendar** from the old app (removed entirely).
- Notifications, reminders, email.
- Export to PDF/Excel/CSV.
- Trend charts / historical analytics.
- A four-type absence system (reduced to two).
- Case sub-types, status pipeline, or per-case assignment.

These may return in a later version; the data model should not block them, but **do not build them now**.

---

## 3. Core Concepts (Read This Before the Data Model)

---

Four concepts, and the relationships between them, are the foundation of the whole app. Misunderstanding the Team ↔ Group relationship will produce a wrong data model.

### 3.1 Department

The whole organization. There is effectively one department in this deployment. It owns the **single weekly goal** (for Beredningar) shown on the landing page.

### 3.2 Team (Sektion)

An organizational/reporting unit named "**Sektion 1**", "**Sektion 2**", .... The department has many, and the number is **unbounded** — coordinators create new sektioner on demand as the department grows (typically once a new team has completed introduction).

- **Case counters live on the team.** Each team has its own Beredningar/Beslut counts per week.
- The manager primarily cares about **per-team progress**; the department total is the sum of all teams.
- Every member belongs to **exactly one team**.

### 3.3 Group

A **work-rotation unit** ("Grupp 1" ... "Grupp 6", plus any manual groups). A group answers the question "**what task do you have this week?**" — nothing more.

- Groups are **mixed across teams**: a single group contains people from different sektioner.
- Every member belongs to **exactly one group**.
- Groups are NOT children of teams.

### 3.4 The Orthogonality Rule (critical)

**Team and Group are two independent ways of slicing the same people. They are orthogonal — neither is nested inside the other.**

- A person has a **teamId** (whose counter their work feeds) **and** a **groupId** (what task they do this week). The two are unrelated.
- The **tracker** is organized by **team**. The **schedule** is organized by **group**. The same person appears in both, grouped differently.

### 3.5 Two Trees

Because of orthogonality, the data splits into two trees that barely interact:

- **Reporting tree:** department → teams → weekly counts → department goal. (The manager's concern.)
- **Scheduling tree:** groups → weekly task rotation → absence. (The weekly-task concern.)

**The rotation never touches the counters.** A member does Beredningar work because their group is rotated onto it, but their `+1` lands on **their team's** counter regardless of what their group is doing. Rotation is **display-only**.

---

## 4. Roles & Permissions

---

Two roles, both **department-wide** in scope.

### 4.1 Coordinator (Admin)

- One or two people, drawn from the department's own members.
- A coordinator **is also a working member**: they sit in a team and a group and do `+/-` work like anyone else. The coordinator role is **additive** — all member powers **plus** admin tools.
- Coordinator status is a `role` field on the user document, not a singleton — there can be one, two, or more without code changes.

### 4.2 Member

- Every other user.
- Can see the entire department (read-only) for the shared overview.
- Can write only: their own team's case counters, and absence (absence is open — anyone's).

### 4.3 Capability Matrix

| Capability                                    | Member | Coordinator |
|-----------------------------------------------|--------|-------------|
| View landing page / department goal           | ✓      | ✓           |
| View any team's numbers (read)                | ✓      | ✓           |
| View full schedule & all absence (read)       | ✓      | ✓           |
| +/- case counts on <b>own</b> team            | ✓      | ✓           |
| +/- case counts on <b>another</b> team        | —      | ✓           |
| Set the weekly department goal (Beredningar)  | —      | ✓           |
| Create / remove a team (sektion)              | —      | ✓           |
| Create / edit / remove a user                 | —      | ✓           |
| Set a user's name, team, group, role          | —      | ✓           |
| Add a manual group                            | —      | ✓           |
| Delete a (non-rotating, empty) group          | —      | ✓           |
| Edit rotation: slots, base, per-week override | —      | ✓           |
| Set a manual group's weekly task              | —      | ✓           |
| Set <b>own</b> absence                        | ✓      | ✓           |
| Set <b>another person's</b> absence           | ✓      | ✓           |
| Use the absence date-range tool               | ✓      | ✓           |
| Appearance / global theme                     | —      | ✓           |
| Personal dark/light toggle (localStorage)     | ✓      | ✓           |

### 4.4 The UI / Security Pairing (do not skip)

Every coordinator-only function appears in the UI **only** when `role == "coordinator"`. **Hiding the button is courtesy; the matching Firestore rule is the actual security.** Both are required. The React app reads the same `role` field the rules read: the UI uses it to *show*, the rules use it to *allow*.

## 5. Tech Stack

| Layer        | Technology                                                                    | Notes                                                                             |
|--------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Framework    | React 18 (functional components + hooks)                                      | Same as old app.                                                                  |
| Build tool   | Vite                                                                          | Keep.                                                                             |
| Language     | JavaScript (JSX)                                                              | TypeScript optional; if adopted, prioritize the user/auth layer (see §13).        |
| Database     | Firebase Firestore                                                            | Real-time via <code>onSnapshot</code> .                                           |
| Auth         | Firebase Authentication                                                       | Email/password. Replaces the old hardcoded PIN.                                   |
| File storage | Firebase Storage                                                              | Background images only.                                                           |
| Routing      | React Router v6                                                               | Deep-linkable pages.                                                              |
| State        | React <code>useState</code> / <code>useEffect</code> + Context for auth/theme | Avoid prop-drilling <code>t</code> / <code>s</code> like the old app did.         |
| Styling      | (your choice — see §13)                                                       | Old app used inline styles; consider CSS Modules or Tailwind for maintainability. |
| Deployment   | Vercel                                                                        | Auto-deploy on git push.                                                          |

UI language is **Swedish**. App must support **dark and light mode** and be **responsive / mobile-friendly** (the old app's tables broke on phones — fix this).

## 6. Firestore Data Model

All data lives under a single department document. Week keys use the format `"YYYY-WW"` with a zero-padded ISO week, e.g. `"2026-23"`. Define this once in a `weekKey(year, weekNum)` helper and use it everywhere — inconsistent key formats were a real bug source in the old app.

```
departments/{deptId}
├── (fields)           { name, createdAt }
├── users/{uid}        { name, role, teamId, groupId, active }
├── teams/{teamId}     { name, order, createdAt }
│   └── weeks/{YYYY-WW} { Beredningar: <int>, Beslut: <int> }
├── goals/{YYYY-WW}    { Beredningar: <int> }
├── groups/{groupId}   { name, order, rotating: <bool>, pairWith: <groupId|null> }
├── config/rotation     { baseWeek, baseYear, slots[], assignments{}, overrides{},
manualTasks{} }
├── config/appearance  { theme, bgImage, bgImageOpacity, customBgColor }
└── absence/{YYYY-WW}  { "{uid}-0": { type, note }, "{uid}-2": { type, note }, ... }
```

## 6.1 Field Reference

`users/{uid}` — the document ID **is** the Firebase Auth uid.

| Field                | Type                     | Notes                                                                      |
|----------------------|--------------------------|----------------------------------------------------------------------------|
| <code>name</code>    | string                   | Set by a coordinator (members can't self-name → people stay identifiable). |
| <code>role</code>    | "coordinator"   "member" | The single source of truth for both UI gating and security rules.          |
| <code>teamId</code>  | string                   | Which sektion's counters this user's work feeds.                           |
| <code>groupId</code> | string                   | Which rotation group this user is in.                                      |
| <code>active</code>  | bool                     | Soft-disable a user without deleting history.                              |

`teams/{teamId}`

| Field                  | Type      | Notes             |
|------------------------|-----------|-------------------|
| <code>name</code>      | string    | e.g. "Sektion 3". |
| <code>order</code>     | int       | Display ordering. |
| <code>createdAt</code> | timestamp | Server timestamp. |

`teams/{teamId}/weeks/{YYYY-WW}` — the case counters.

| Field                    | Type    | Notes                                 |
|--------------------------|---------|---------------------------------------|
| <code>Beredningar</code> | int ≥ 0 | Has a department goal + progress bar. |
| <code>Beslut</code>      | int ≥ 0 | Goal-less running tally.              |

`goals/{YYYY-WW}` — the department weekly target.

| Field                    | Type | Notes                                                                                                                                 |
|--------------------------|------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>Beredningar</code> | int  | <b>Only</b> Beredningar. Beslut has no goal. Set fresh each week by a coordinator; document is <b>absent until set</b> (blank state). |

`groups/{groupId}`

| Field                 | Type           | Notes                                                                                                |
|-----------------------|----------------|------------------------------------------------------------------------------------------------------|
| <code>name</code>     | string         | e.g. "Grupp 1".                                                                                      |
| <code>order</code>    | int            | Display ordering.                                                                                    |
| <code>rotating</code> | bool           | <code>true</code> for the original paired groups (frozen set); <code>false</code> for manual groups. |
| <code>pairWith</code> | groupId   null | The partner group in the rotation (e.g. Grupp 1 ↔ Grupp 2). <code>null</code> for manual groups.     |

`config/rotation` — see Section 8 for full semantics.

| Field                                            | Type     | Notes                                                                                                                                 |
|--------------------------------------------------|----------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>baseWeek</code> ,<br><code>baseYear</code> | int      | The reference week the rotation is computed from ( <b>this week</b> at launch).                                                       |
| <code>slots</code>                               | string[] | The three editable task labels, by index.                                                                                             |
| <code>assignments</code>                         | map      | Base mapping <code>pairKey</code> → <code>slotIndex</code> for rotating pairs.                                                        |
| <code>overrides</code>                           | map      | <code>weekKey</code> → { <code>pairKey</code> : <code>slotIndex</code> } — per-week exceptions that DON'T shift the ongoing rotation. |
| <code>manualTasks</code>                         | map      | <code>weekKey</code> → { <code>groupId</code> : <string task> } — hand-set tasks for non-rotating groups.                             |

`absence/{YYYY-WW}` — one flat document per week (like the old app).

| Key                                 | Value                                        | Notes                                                                                                                                                                                                                                                                          |
|-------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>"{uid}-0" .. "{uid}-4"</code> | { <code>type</code> ,<br><code>note</code> } | Key = uid + weekday index (0 = Mon ... 4 = Fri). <code>type</code> ∈ <code>"semester"</code>   <code>"ovrig"</code> . <code>note</code> = optional partial-day text ("fm", "em", "2h", "09-11"). A missing key = present that day. <b>Open: any member may read and write.</b> |

## 6.2 Why the model is shaped this way (rationale to preserve)

- **Counts on teams, goal on department** mirrors how the work is reported: per-team progress summing to a department total.
- **Goal is per-week** ( `goals/{YYYY-WW}` ) because the coordinator updates it weekly — it is time-series data, not a static setting.
- **uid-keyed users and absence** mean renaming a person never orphans data (the key carries the uid, not the name).
- **Absence is one flat document per week, open to all members** — anyone can fill in anyone's vacation/absence. The structure is flat (not split per member) because there is no per-key ownership rule to satisfy; writes use merge / field paths so concurrent edits to the same week don't clobber each other (see §7.5).
- **Groups top-level, not under teams** encodes the orthogonality rule.
- **Personal dark/light in `localStorage`, not Firestore** keeps members at zero Firestore write access for preferences.

## 7. Feature Specifications

### 7.1 Authentication & Onboarding

- Firebase Auth, email + password.
- **No self-registration.** Coordinators create user accounts and set each user's `name` , `teamId` , `groupId` , and `role` .
- **Bootstrap:** the very first coordinator is created **manually in the Firebase console** (Auth user + matching `users/{uid}` doc with `role:"coordinator"` ). Every coordinator after that is created in-app by an existing coordinator.

- On login, the app loads the user's own `users/{uid}` doc and derives an `isCoordinator` flag, used to gate all admin UI.

## 7.2 Landing Page / Department Dashboard

The shared overview. For the **current week** it shows:

- **Beredningar goal widget** — target (from `goals/{YYYY-WW}`) vs. handled (sum of every team's `weeks/{YYYY-WW}.Beredningar`). Shows a progress bar, e.g. "18 av 25 · 7 kvar."
- **Blank state:** if no goal doc exists for the week, show a calm "Inget mål satt än" and an inert bar. This doubles as a gentle reminder to coordinators. **No carry-forward** from prior weeks.
- **Beslut tally** — plain total handled across all teams (sum of `weeks/{YYYY-WW}.Beslut`). **No goal, no progress bar.**
- The page **subscribes** to the `teams` collection (so a newly added team appears live) and to each team's current-week doc.
- Coordinator-only: an inline control to set/update this week's Beredningar goal.

**Scale note:** the rollup is "read all teams + sum," i.e. N reads where N = team count. Fine up to dozens of teams. If the department ever exceeds ~50 teams, revisit with a maintained aggregate counter. **Do not pre-optimize this now.**

## 7.3 Team Tracker Page (per Sektion)

Each team has its own tracker page:

- **Beredningar** and **Beslut** counters for the selected week, each with `+` / `-` buttons.
- Writes use a **Firestore transaction** (`runTransaction`) so two users clicking simultaneously never lose an increment.
- **Member access:** a member sees all teams but can only press `+/-` on **their own** team; other teams' controls are read-only/disabled (and the rule enforces it).
- **Coordinator access:** may adjust any team, and may type an exact value directly.
- Week navigation (see §7.7) lets any week be viewed; historical weeks are read back from their week-keyed docs.

## 7.4 Schedule Page (Schema)

Organized by **group**, laid out in **two columns**.

- Shows each group, the members in it (mixed across teams), and the group's **task for the selected week**.
- **Rotating groups:** task is **computed** from the rotation (Section 8). The three rotating pairs cycle through the three slots automatically each week — nobody has to set it.
- **Manual groups:** task is **read from** `manualTasks`; if unset for that week, show "—" (honest empty state) until a coordinator sets it.
- **Coordinator-only controls:** edit the three slot labels, set a per-week override for a rotating pair, set a manual group's weekly task, add a manual group, delete a (non-rotating, empty) group, reassign members between groups.

## 7.5 Absence Page (Frånvaro)

- Per-member, Mon–Fri grid for the selected week.
- **Two absence types only:** **Semester** and **Övrig**. (Down from the old four.)



- Each set cell reveals an inline **note** field for partial-day text ("fm", "em", "2h", "09-11"). **Note saves on blur, not per keystroke**, to avoid Firestore write spam.
- **Open access:** absence is **fully open** — any member may read and write anyone's absence. There is no ownership restriction in the UI or the rules. (The team wants anyone to be able to fill in a colleague's vacation.)
- **Date-range tool (for everyone):** a prominent "**Frånvaroperiod**" action lets any user mark a person absent across a **start date → end date** in one operation, instead of clicking day by day. This is the easy path for long vacations.
- Implementation: iterate every calendar day in the range, group days by their ISO week, and write each affected week's absence document. Use **merge writes / field paths** (or a transaction per week) so the range write only touches the targeted `"{uid}-{day}"` keys and never clobbers other people's entries already in that week.
- The tool sets a single **type** (Semester or Övrig) across the whole range; weekends (Sat/Sun) are skipped since the grid is Mon–Fri.
- A **legend** shows the colour for each absence type.

**Concurrency note:** because absence is one flat document per week and is openly writable, every write — single cell, note, or range — must use merge / field-path writes rather than overwriting the whole week document, or simultaneous editors will erase each other's entries.

## 7.6 Coordinator Admin Functions (surfaced only when `role == "coordinator"`)

- **Add / remove a team (sektion).** Block removing a team that still has members or week data (app-side guard).
- **Create / edit / remove a user;** set name, team, group, role (incl. promoting another member to coordinator).
- **Set the weekly department goal.**
- **Manage groups:** add manual group; delete a non-rotating, empty group; reassign members.
- **Edit rotation:** slot labels, per-week override, manual-group task.
- **Appearance:** global theme, background colour, background image upload (Firebase Storage) with opacity slider.

## 7.7 Global / App-Level

- **Week/year navigation:** arrows + dropdowns to view any week (e.g.  $\pm 3$  years). All pages share the active `{ weekNum, year }`.
- **Real-time sync** on every live view via `onSnapshot`.
- **Save indicator:** brief "*Sparad!*" confirmation after a write.
- **Personal dark/light toggle:** stored in `localStorage` (per-device, not in Firestore).
- **Global theme / background:** coordinator-set, applies to everyone, from `config/appearance`.

# 8. Rotation Logic (Detailed)

## 8.1 Model

- Rotating groups are paired: **(Grupp 1+2), (Grupp 3+4), (Grupp 5+6)** — three pairs.
- There are **three task slots**, each a label in `slots[]`. At launch (from this week):

- slot 0: "**Beredning**"
- slot 1: "**Beslut**"
- slot 2: "**Rekrytera + sökandesammanställningar + Beredningar**"
- **Slot 2 is a bundle that itself includes Beredningar**, so in any week **two pairs touch Beredningar** and one does Beslut. This is fine because rotation is display-only.
- Slot labels are **coordinator-editable** data (stored in `slots[]`), not hardcoded.

## 8.2 Rotation is display-only

The rotation tells a group what they're *responsible* for that week. It **never gates which counter a member may touch**. Any member may `+/-` any category on their own team regardless of their group's slot.

## 8.3 The formula

Period = 3 (three pairs, three slots). The whole rotation is one number: weeks elapsed since the base, mod 3.

```
function getAssignmentsForWeek(year, weekNum, rotation) {
  const key = weekKey(year, weekNum);          // "2026-23"

  // Per-week exception: use stored assignment verbatim, do NOT shift the cycle.
  if (rotation.overrides && rotation.overrides[key]) {
    return rotation.overrides[key];             // { pairKey: slotIndex }
  }

  const elapsed = weeksBetween(
    rotation.baseYear, rotation.baseWeek, year, weekNum
  );
  const shift = ((elapsed % 3) + 3) % 3;         // guard against negative

  // shift each pair's base slotIndex forward by `shift`, mod 3
  const result = {};
  for (const pairKey in rotation.assignments) {
    result[pairKey] = (rotation.assignments[pairKey] + shift) % 3;
  }
  return result;
}
```

Two non-obvious correctness points (both bit the old app's domain):

- **weeksBetween must count ISO weeks across year boundaries**. Don't do `weekNum - baseWeek` (week 52 → week 1 is +1, not -51, and some ISO years have 53 weeks). The robust way: convert each `(year, week)` to its **Monday Date** via `getMondayOfWeek`, then divide the day-difference by 7.
- **The negative-modulo guard** `((x % 3) + 3) % 3` matters because users navigate to **past** weeks; JS `%` returns negatives for negative inputs, producing a wrong slot for any week before the base.

## 8.4 Overrides

A coordinator override is a **per-week exception**: it sets the assignment for that one week and the ongoing rotation **keeps computing from the base** for all other weeks (no re-base). Stored in `overrides[weekKey]`.

## 8.5 Manual groups

- The rotating set is **frozen** — coordinators can never add to it (an odd number of groups would break pairing). They may only add **manual** groups (`rotating:false`).

- A manual group's weekly task is **stored** in `manualTasks[weekKey][groupId]`, set by hand by a coordinator. Unset = display "—".
- The schedule's per-group task lookup:

```
function taskForGroup(group, year, weekNum, rotation, slots) {
  if (group.rotating) {
    const a = getAssignmentsForWeek(year, weekNum, rotation);
    const pairKey = pairKeyForGroup(group);          // e.g. "pair_1_2"
    return slots[a[pairKey]];
  }
  const key = weekKey(year, weekNum);
  return (rotation.manualTasks?.[key]?.[group.id]) ?? "—";
}
```

## 9. Security Model

The complete, deployable ruleset is in `firestore.rules`. Summary of the posture:

- **Read:** any signed-in member of the department may read everything (shared overview).
- **Write:** a member may write (a) their **own team's** weekly counter and (b) **any** absence (absence is intentionally open to all members). Everything else is **coordinator-only**.
- Every coordinator-only **UI** action has a matching coordinator-only **rule**. The UI gate is courtesy; the rule is the enforcement.
- `role` on the user doc is the trust anchor. Because `users/*` is coordinator-write-only, **a member can never promote themselves** — so the `isCoordinator` check can't be forged.
- **Bootstrap** the first coordinator by hand in the console; rules never solve bootstrap.

**Accepted risk (v1):** the member counter rule permits setting their own team's count to any non-negative integer, not strictly "previous  $\pm 1$ ". Acceptable for a small trusted team. Optional hardening: clamp the delta on update with `request.resource.data.Beredningar <= resource.data.Beredningar + N`, but handle the create case (no `resource.data`) carefully. Not needed now.

## 10. Real-Time & Concurrency Patterns (Keep From Old App)

These patterns from the previous app are proven — carry them forward.

**Subscription wrapper** — every live view subscribes and cleans up on unmount / week change:

```
useEffect(() => {
  const unsub = subscribe(path, (data) => setState(data ?? fallback));
  return () => unsub();
}, [weekNum, year]);
```

**Atomic counter writes** — all `+/-` go through a transaction so concurrent clicks don't clobber each other:

```
await adjust(teamWeekPath, (current) => {
  const data = current ?? { Beredningar: 0, Beslut: 0 };
  return { ...data, Beredningar: data.Beredningar + 1 };
});
```

**Save-on-blur for text** — absence notes (and any free-text) write on blur, not per keystroke.

**Default-config fallback** — if a doc doesn't exist yet, fall back to a sensible default in code so a fresh week/team works with zero setup.

## 11. Key Utilities

**Date utilities ( `dateUtils.js` )**

| Function                                    | Purpose                                                           |
|---------------------------------------------|-------------------------------------------------------------------|
| <code>getWeekNumber(date)</code>            | ISO 8601 week number (week starts Monday; week 1 contains Jan 4). |
| <code>getMondayOfWeek(year, weekNum)</code> | Monday <code>Date</code> for a given ISO year + week.             |
| <code>weeksBetween(y1, w1, y2, w2)</code>   | Signed ISO-week distance (via Monday <code>Date</code> diff ÷ 7). |
| <code>toDateKey(date)</code>                | "YYYY-MM-DD" string.                                              |

**Key utilities ( `keys.js` )** — single source of truth for all Firestore keys

| Function                            | Returns                       |
|-------------------------------------|-------------------------------|
| <code>weekKey(year, weekNum)</code> | "2026-23" (zero-padded week). |
| <code>pairKeyForGroup(group)</code> | "pair_1_2" etc.               |

**Data utilities ( `dataUtils.js` )**

| Function                       | Purpose                                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>makeEmptyWeek()</code>   | { <code>Beredningar</code> : 0, <code>Beslut</code> : 0 }.                                                        |
| <code>defaultRotation()</code> | Initial <code>slots</code> , <code>assignments</code> , empty <code>overrides</code> / <code>manualTasks</code> . |

## 12. UX States & Edge Cases

| Situation                                        | Behaviour                                                                                                                             |
|--------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| New week, no goal set                            | Beredningar widget shows "Inget mål satt än", inert bar. No carry-forward.                                                            |
| Manual group, no task set for week               | Show "—".                                                                                                                             |
| Member views another team                        | Numbers visible (read), <code>+/-</code> disabled.                                                                                    |
| Member tries to write another team (console)     | <b>Rejected by rule.</b>                                                                                                              |
| Member moves team mid-week                       | Past contributions stay on the old team's historical week docs; new work counts to the new team. (Automatic — counts are team-keyed.) |
| Delete a group that has members                  | <b>Blocked</b> (app guard). Coordinator must empty it first.                                                                          |
| Delete a rotating group                          | <b>Blocked</b> — rotating set is frozen.                                                                                              |
| Delete a non-rotating, empty group               | Allowed.                                                                                                                              |
| Delete a team with members/data                  | <b>Blocked</b> (app guard).                                                                                                           |
| Navigate to a week before the rotation base      | Formula's negative-modulo guard returns the correct historical assignment.                                                            |
| Absent member's day                              | A <code>"{uid}-{day}"</code> key present in the week's absence doc; missing key = present.                                            |
| Two people edit absence in the same week at once | Both writes use merge / field paths, so they don't clobber each other.                                                                |

## 13. Non-Functional Requirements

- **Responsive / mobile-first.** The old table layout broke on phones — the tracker and schedule must be usable on a narrow screen.
- **Swedish UI** throughout.
- **Dark + light mode**, personal preference in `localStorage`; global theme/background coordinator-set.
- **Theme as tokens** — keep the token concept from the old app (a set of named colour tokens), but deliver it via React Context or CSS custom properties instead of prop-drilling `t / s` into every component.
- **Maintainable styling** — prefer CSS Modules or Tailwind over inline styles.
- **State via Context** for auth and theme; local component state otherwise.

**TypeScript (optional but recommended for two layers):** if adopted, prioritize (1) the **user/auth record** ( `role` , `teamId` , `groupId` — referenced by every permission check; a typo is a silent bug) and (2) the **week-key helpers**. A single typed `keys.js` / `user` shape captures most of the safety even in plain JS.

## 14. Suggested File Structure

```
src/
  main.jsx          React entry (ReactDOM.createRoot)
  App.jsx           Router, auth gate, week state, theme context

  context/
    AuthContext.jsx  Firebase Auth + current user doc + isCoordinator
    ThemeContext.jsx Theme tokens, dark/light, global appearance

  firebase/
    firebase.js      init; exports auth, db, storage
    storage.js       get / set / subscribe / adjust(transaction) / uploadImage

  utils/
    dateUtils.js     week/date helpers
    keys.js           weekKey, pairKeyForGroup – single source of truth
    dataUtils.js     empty-week, default rotation, fallbacks
    rotation.js       getAssignmentsForWeek, taskForGroup

  pages/
    DashboardPage.jsx department goal + rollups (landing)
    TeamTrackerPage.jsx per-team Beredningar/Beslut +/-
    SchedulePage.jsx   two-column group schedule + rotation
    AbsencePage.jsx    Mon–Fri absence grid + date-range tool (open)

  components/
    GoalWidget.jsx    Beredningar target vs handled + bar
    BeslutTally.jsx   goal-less total
    Counter.jsx       +/- with transaction
    GroupCard.jsx     one group, its members + weekly task
    AbsenceGrid.jsx   Mon–Fri cells + on-blur note
    WeekPicker.jsx    arrows + year/week dropdowns
    admin/
      TeamManager.jsx coordinator-only panels
      UserManager.jsx
      GroupManager.jsx
      RotationEditor.jsx
      GoalEditor.jsx
      AppearancePanel.jsx

  firestore.rules     the deployable security rules (companion file)
```

## 15. Build Order (Suggested)

1. Firebase project + Auth + Firestore; deploy `firestore.rules`; seed the first coordinator by hand.
2. Auth gate + `AuthContext` (login, load own user doc, `isCoordinator`).
3. Data layer ( `storage.js` : subscribe / adjust / set / get ) + `keys.js` + `dateUtils.js`.
4. Week picker + routing shell.
5. Team tracker (counters + transactions) — the core daily action.
6. Dashboard rollup (goal widget + Beslut tally), live-subscribed to teams.
7. Schedule + rotation ( `rotation.js` ), rotating groups first, then manual groups.
8. Absence (flat per-week doc, open writes with merge, on-blur notes, date-range tool).

- 9. Coordinator admin panels (teams, users, groups, goal, rotation, appearance).
  - 10. Theme/Context, responsive polish, dark/light.
- 

## 16. Summary of Decisions (Quick Reference)

---

- **Two roles:** Member, Coordinator (additive admin). 1–2 coordinators, department-wide.
  - **Teams (sektioner)** are unbounded, coordinator-created. **Counts live on teams.**
  - **Department goal** is weekly, coordinator-set, **Beredningar only, blank until set.**
  - **Beslut** is a goal-less tally that still rolls up.
  - **Groups** are orthogonal to teams; one team + one group per member.
  - **Rotation:** 3 pairs × 3 slots, period 3, computed from a base week; **display-only**; per-week overrides don't shift the cycle; slot labels editable.
  - **Rotating group set is frozen**; manual groups are addable, hand-tasked, deletable only when empty.
  - **Absence:** two types (Semester, Övrig), one flat doc per week, **open to all members** (anyone can edit anyone), on-blur notes, and a **date-range tool** for long vacations. Merge writes to avoid clobbering.
  - **Security:** broad-read, strict-write; everything coordinator-only except own-team counter and absence (absence open to all members); first coordinator seeded by hand.
  - **Removed from old app:** decision-deadline calendar, two of four absence types, shared admin PIN.
  - **Out of scope v1:** manager role, per-team coordinators, notifications, export, charts.
- 

Build spec for the next-generation department dashboard. Companion file: `firestore.rules`.